

Lecture 19: Implementation and zero-stability of multistep methods

Jamie Haddock

Table of contents

1	Implementation of multistep methods	1
1.1	Explicit methods	1
1.2	Example	2
1.3	Implicit methods	3
1.4	Stiff problems	4
1.5	Example	4
2	Zero-stability of multistep methods	7
2.1	Example	7
2.2	Zero-stability	9
2.3	Dahlquist theorems	9
2.4	Accuracy vs Stability	10

1 Implementation of multistep methods

We now consider some of the practical issues that arise when multistep formulas are used to solve IVPs.

1.1 Explicit methods

Consider the AB4 method:

$$\mathbf{u}_{i+1} = \mathbf{u}_i + h \left(\frac{55}{24} \mathbf{f}_i - \frac{59}{24} \mathbf{f}_{i-1} + \frac{37}{24} \mathbf{f}_{i-2} - \frac{9}{24} \mathbf{f}_{i-3} \right), \quad i = 3, \dots, n-1.$$

```
"""
    ab4(ivp,n)

Apply the Adams-Bashforth 4th order method to solve the given IVP
using `n` time steps. Returns a vector of times and a vector of
solution values.
"""
function ab4(ivp,n)
    # Time discretization.
    a,b = ivp.tspan
    h = (b-a)/n
    t = [ a + i*h for i in 0:n ]

    # Constants in the AB4 method.
    k = 4;      = [55,-59,37,-9]/24;

    # Find starting values by RK4.
    u = fill(float(ivp.u0),n+1)
```

```

rkivp = ODEProblem(ivp.f, ivp.u0, (a, a+(k-1)*h), ivp.p)
ts, us = rk4(rkivp, k-1)
u[1:k] .= us

# Compute history of u' values, from newest to oldest.
f = [ ivp.f(u[k-i], ivp.p, t[k-i]) for i in 1:k-1 ]

# Time stepping.
for i in k:n
    f = [ ivp.f(u[i], ivp.p, t[i]), f[1:k-1]... ] # new value of du/dt
    u[i+1] = u[i] + h*sum(f[j]*[j] for j in 1:k) # advance a step
end
return t, u
end

```

ab4

1.2 Example

We study the convergence of AB4 using the IVP $u' = \sin[(u+t)^2]$ over $0 \leq t \leq 4$, with $u(0) = -1$. As usual, `solve` is called to give an accurate reference solution.

rk4

```

ivp = ODEProblem((u,p,t)->sin((t+u)^2), -1., (0.0, 4.0))
u_ref = solve(ivp, Tsit5(), reltol=1e-14, abstol=1e-14);

```

Now we perform a convergence study of the AB4 code.

```

n = @. [ round(Int, 4*10^k) for k in 0:0.5:3 ]
errs = []
for n in n
    t, u = ab4(ivp, n)
    push!( errs, norm(u_ref.(t)-u, Inf) )
end

pretty_table([n errs], header=["n", "inf-norm error"])

```

n	inf-norm error
4	0.50044
13	1.39129
40	0.00627809
126	9.94942e-5
400	1.09598e-6
1265	1.12766e-8
4000	1.13736e-10

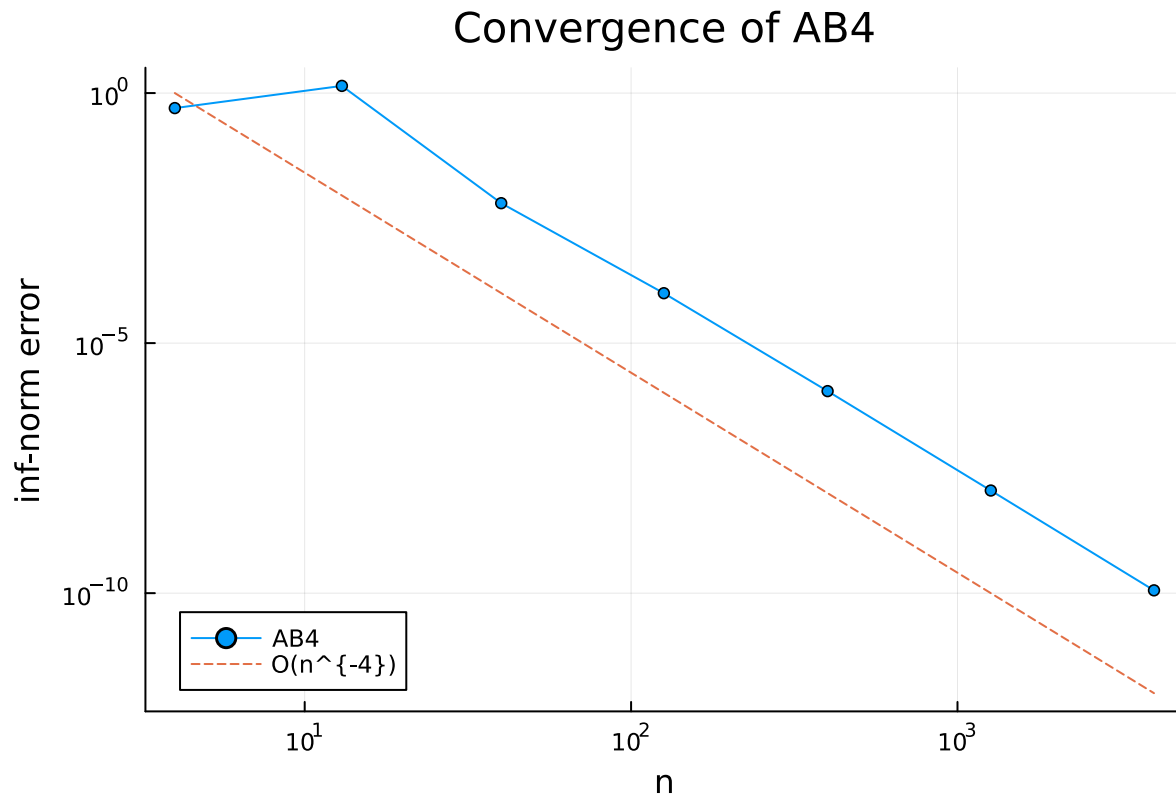
The method should converge as $O(h^4)$, so a log-log scale is appropriate for the errors.

```

plot(n, errs, m=3, label="AB4",
     xaxis=(log10, "n"), yaxis=(log10, "inf-norm error"),
     title="Convergence of AB4", leg=:bottomleft)

```

```
plot!(n, (n/n[1]).^(-4), l=:dash, label="O(n^{-4})")
```



fdjac

levenberg

1.3 Implicit methods

Implementing an implicit multistep method is more difficult. Consider the second-order implicit formula AM2, also known as the trapezoid method. To advance from step i to $i + 1$, we need to solve for $\mathbf{z}$ in the equation

$$\mathbf{z} - \frac{1}{2}h\mathbf{f}(t_{i+1}, \mathbf{z}) = \mathbf{u}_i + \frac{1}{2}h\mathbf{f}(t_i, \mathbf{u}_i)$$

This equation can be written as $\mathbf{g}(\mathbf{z}) = 0$, so the rootfinding methods of Chapter 4 can be used. The new value $\mathbf{u}_{i+1}$ is equal to the root of this equation.

```
"""
    am2(ivp,n)

Apply the Adams-Moulton 2nd order method to solve given IVP using
`n` time steps. Returns a vector of times and a vector of
solution values.
"""
function am2(ivp,n)
    # Time discretization.
    a,b = ivp.tspan
```

```

h = (b-a)/n
t = [ a + i*h for i in 0:n ]

# Initialize output.
u = fill(float(ivp.u0),n+1)

# Time stepping.
for i in 1:n
    # Data that does not depend on the new value.
    known = u[i] + h/2*ivp.f(u[i],ivp.p,t[i])
    # Find a root for the new value.
    g = z -> z - h/2*ivp.f(z,ivp.p,t[i+1]) - known
    unew = levenberg(g,known)
    u[i+1] = unew[end]
end
return t,u
end

```

am2

1.4 Stiff problems

At each time step, an implicit IVP solver requires a rootfinding iteration, requiring multiple calls to evaluate the function f . This fact makes the cost of an implicit method much greater on a per-step basis than an explicit one.

Given this drawback, you are justified to wonder whether implicit methods are ever competitive!

The answer is emphatically yes!

1.5 Example

The following simple ODE uncovers a surprise.

```
ivp = ODEProblem((u,p,t)->u^2-u^3, 0.005, (0,400.))
```

```

ODEProblem with uType Float64 and tType Float64. In-place: false
timespan: (0.0, 400.0)
u0: 0.005

```

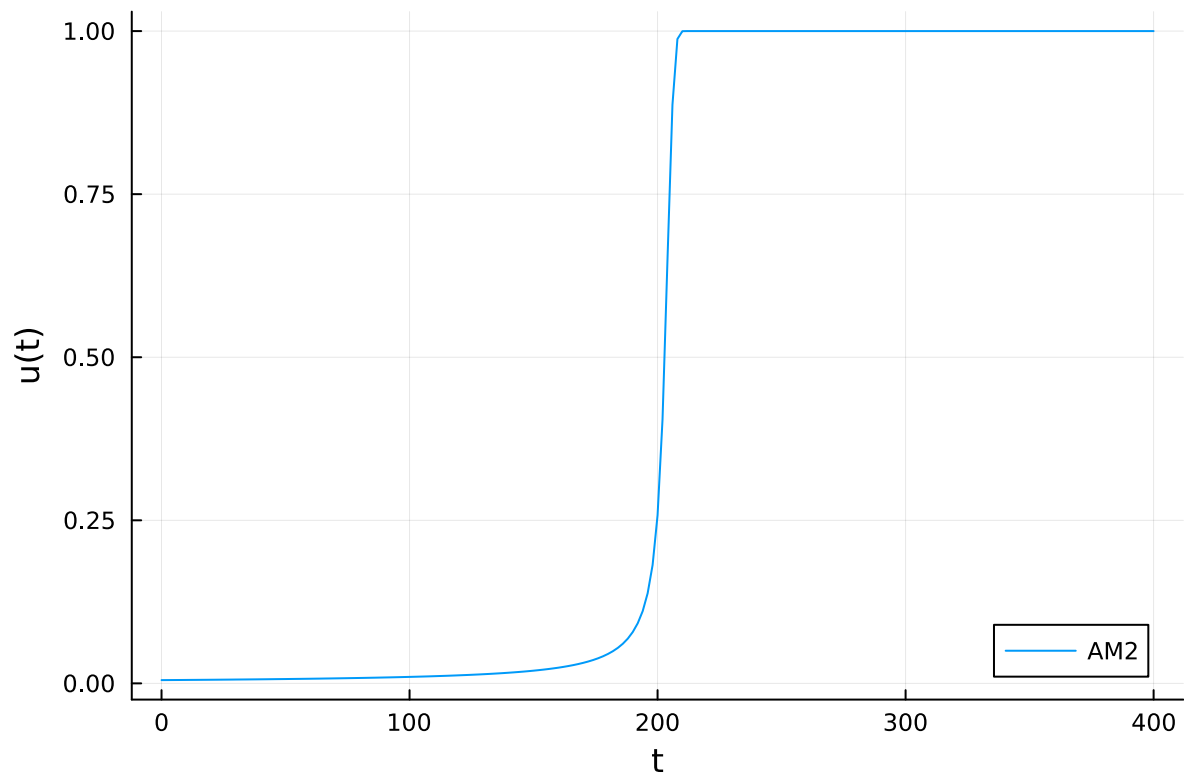
We will solve the problem first with the implicit AM2 method using $n = 200$ steps.

```

tI,uI = am2(ivp,200)

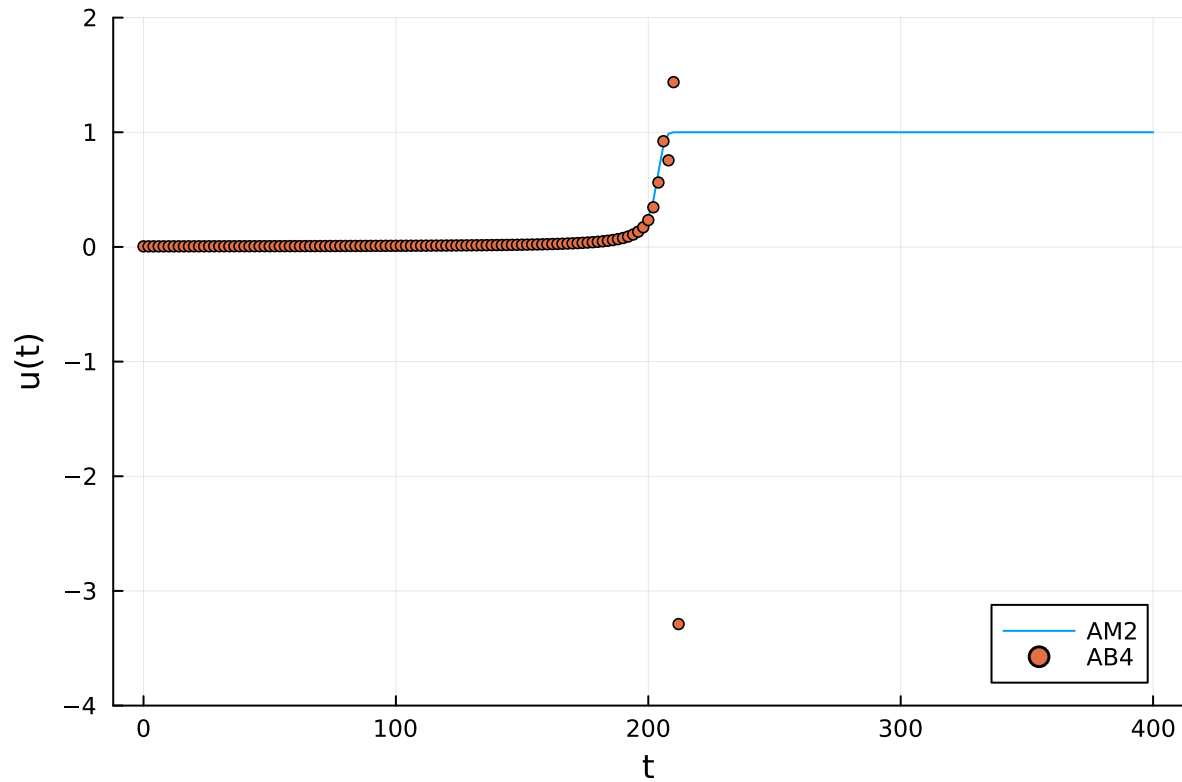
plot(tI,uI,label="AM2",
     xlabel="t",ylabel="u(t)",leg=:bottomright)

```



Now we repeat the process using the explicit AB4 method.

```
tE,uE = ab4(ivp,200)
scatter!(tE,uE,m=3,label="AB4",ylim=[-4,2])
```

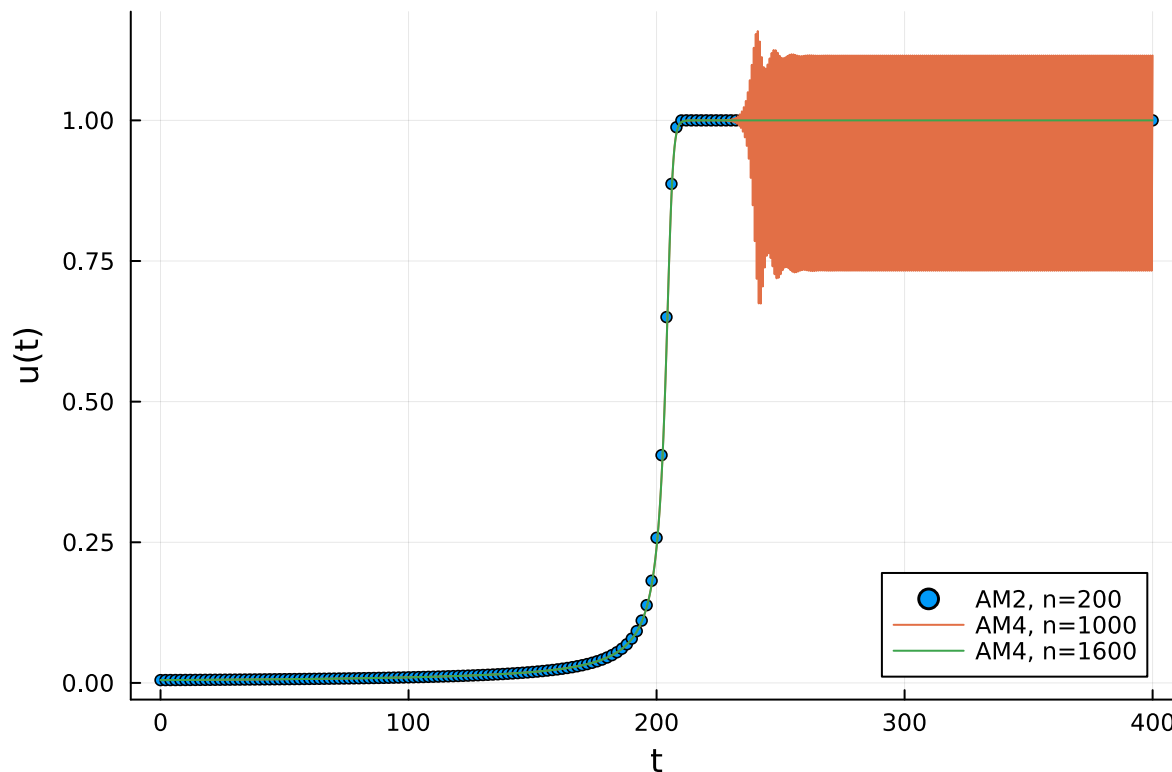


Once the solution starts to take off, the AB4 result goes catastrophically wrong.

We hope that AB4 will converge in the limit $h \rightarrow 0$, so let's try using more steps.

```
plt = scatter(tI,uI,label="AM2, n=200",m=3,
             xlabel="t",ylabel="u(t)",leg=:bottomright)

for n in [1000,1600]
    tE,uE = ab4(ivp,n)
    plot!(tE,uE,label="AM4, n=$n")
end
plt
```



So AB4, which is supposed to be *more* accurate than AM2, actually needs something like 8 times as many steps to get a reasonable-looking answer!

This may seem like a contradiction to our understanding of orders of accuracy, but it is not.

A fourth-order explicit formula is more accurate than a second-order implicit one, in the limit $h \rightarrow 0$. But there is another limit to consider, $t \rightarrow \infty$ with h fixed, and in this one the implicit method wins!

Problems for which implicit methods are much more efficient than explicit counterparts are called **stiff**. A sure sign of stiffness is the presence of phenomena on widely different time scales.

In this example, there are two slow periods during which the solution changes very little, interrupted by a very fast transition in the state. An explicit method “thinks” that the step size must always be dictated by the time scale of the fast transition, whereas an implicit method can take large steps during the slow periods.

2 Zero-stability of multistep methods

For one-step methods such as Runge–Kutta, it is guaranteed that the method converges and that the global error is of the same order as the local truncation error.

For multistep methods, however, a new wrinkle is introduced.

2.1 Example

Once can check that the two-step method LIAF, given by

$$\mathbf{u}_{i+1} = -4\mathbf{u}_i + 5\mathbf{u}_{i-1} + h(4\mathbf{f}_i + 2\mathbf{f}_{i-1}),$$

is third-order accurate.

Let's apply it to the ridiculously simple IVP $u' = u$, $u(0) = 1$, whose solution is e^t . We'll measure the error at the time $t = 1$.

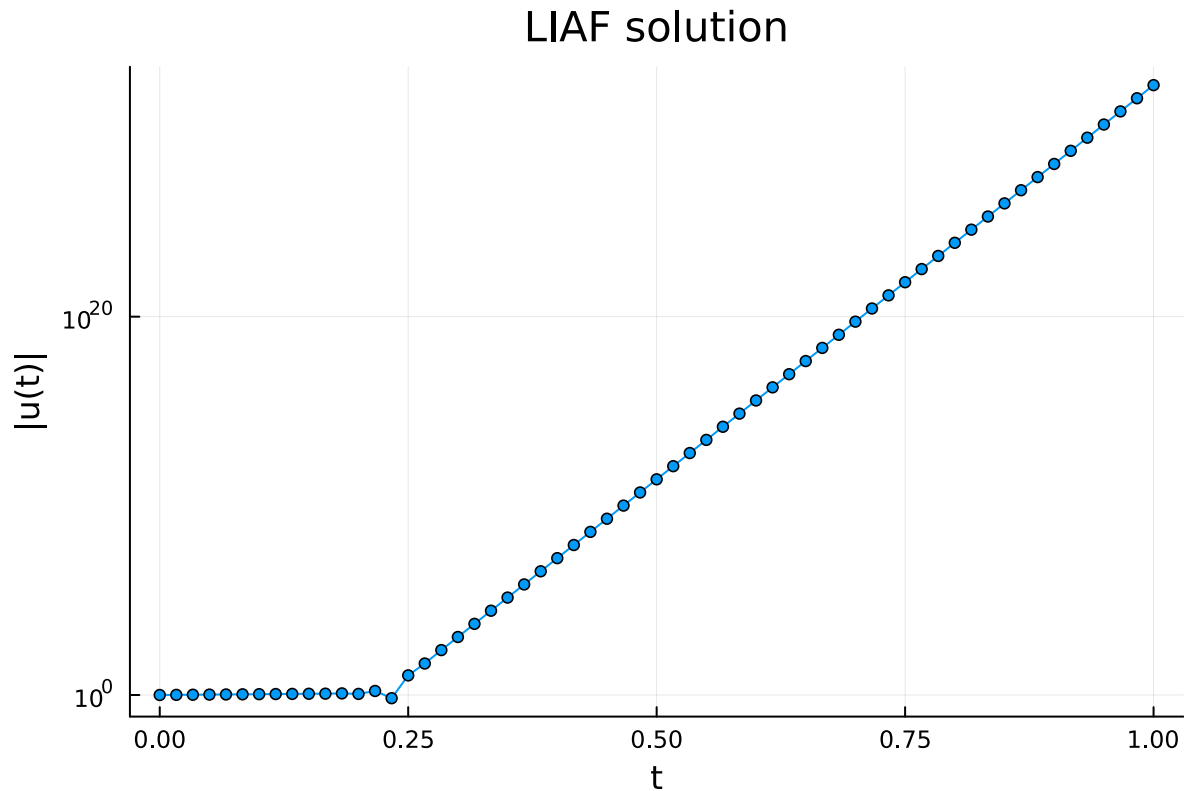
```
dudt = (u,t) -> u
û = exp
a,b = 0.0,1.0;
n = [5,10,20,40,60]
errs = []
t,u = [],[]
for n in n
    h = (b-a)/n
    t = [ a + i*h for i in 0:n ]
    u = [1; û(h); zeros(n-1)]
    f = [dudt(u[1],t[1]); zeros(n)]
    for i in 2:n
        f[i] = dudt(u[i],t[i])
        u[i+1] = -4*u[i] + 5*u[i-1] + h*(4*f[i]+2*f[i-1])
    end
    push!( errs, abs(û(b) - u[end]) )
end

pretty_table( [n (b-a)./n errs], header=["n","h","error"] )
```

n	h	error
5	0.2	0.0160452
10	0.1	2.84548
20	0.05	1.6225e6
40	0.025	9.3442e18
60	0.0166667	1.74013e32

The error starts out promisingly, but things explode from there. A graph of the last numerical attempt yields a clue.

```
plot(t,abs.(u),m=3,label="",
     xlabel="t",yaxis=(log10,"|u(t)|"),title="LIAF solution")
```

It's clear that the solution is growing exponentially in time.

2.2 Zero-stability

Here is the crucial property that LIAF lacks.

Definition: Zero-stability of a multistep IVP method

A multistep method is **zero-stable** if, as $h \rightarrow 0$, every numerical solution produced by the method remains bounded throughout $a \leq t_i \leq b$.

Without zero-stability, any truncation or roundoff error will get exponentially amplified and eventually overwhelm convergence to the exact solution.

2.3 Dahlquist theorems

It turns out that lacking zero-stability is the only thing that can go wrong for a consistent multistep method.

Theorem: Dahlquist equivalence

A linear multistep method converges as $h \rightarrow 0$ if and only if it is consistent and zero-stable.

The Dahlquist equivalence theorem is one of the most important and celebrated in the history of numerical analysis. It can be proved more precisely that a zero-stable, consistent method is convergent in the same sense as Euler's method, with the error between numerical and exact solutions being of the same order as the local truncation error, for a wide class of problems.

2.4 Accuracy vs Stability

You may have noticed that the Adams and BD formulas use only about half of the available data from the past k steps, i.e., they have many possible coefficients set to zero. For instance, a k -step AB method uses only the f_j -values and has order k .

The order could be made higher by also using u_j -values, like the LIAF method does for $k = 2$. Also like the LIAF method, however, such attempts are doomed by instability.

Theorem: First Dahlquist stability barrier

The order of accuracy p of a stable k -step linear multistep method satisfies

$$p \leq \begin{cases} k + 2 & \text{if } k \text{ is even,} \\ k + 1 & \text{if } k \text{ is odd,} \\ k & \text{if the method is explicit.} \end{cases}$$

The lesson of the result is that accuracy is not the only important feature, and trying to optimize for it leads to failure.